

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Tema 3 – Networking

Conceptos básicos: lectura de repaso

- Please, give me your address
- maxpower@springfield.org
- Not this, the other...
- 80.58.0.33
- No, the private, please
- 192.168.1.11
- I mean the physic one!
- 00:1d:7d:e5:91:a0
- OMG, home address!
- 127.0.0.1

Si no entiendes lo anterior hay muuucho que repasar...

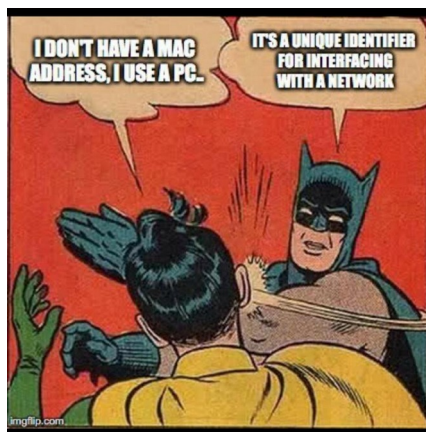
Para poder realizar programación en redes, lo primero es tener claro las distintas arquitecturas y conceptos usados en el mundo del networking tal cual se vio en la asignatura de primero de Sistemas Informáticos. En cualquier caso haremos un rápido repaso a los conceptos esenciales.

En una red tenemos una serie de equipos (**hosts**) interconectados entre sí formando a su vez subredes y a la vez enrutados e interconectados mediante distintos dispositivos como **routers** y **switches**.

Estos equipos se comunican entre sí enviándose **paquetes**, que son secuencias de bytes con la información que se quiere transmitir además de información de control que indica a la red qué hacer con el paquete (por ejemplo su destino).

Por tanto, un paquete contiene tanto la información útil (**datos de la aplicación**) como información de control (**cabeceras o headers**) que añade cada capa del modelo de red (como direcciones, checksums, números de secuencia) para que el enrutamiento y la comunicación sean posibles.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					



Protocolos de comunicación

Un **protocolo de comunicación** es un convenio mediante el cual dos máquinas pueden iniciar y llevar a cabo una comunicación. Para ello el protocolo define una secuencia determinada de paquetes, los tipo de paquetes, su tamaño, que comandos de control incluyen, qué significan los bytes de control y como están contruidos, etc...

Existen distintos protocolos para optimizar distintos tipos de comunicaciones. Por ejemplo http para intercambio de páginas web, ftp para intercambio de archivos, etc...

Una de las tareas principales cuando se programa una aplicación con conexión es establecer un protocolo de comunicación: o se coge uno existente o hay que crear uno nuevo.

TCP/IP es un conjunto de protocolos en capas en el que está basado Internet. Los principales son **TCP** (Transmission Control Protocol), **UDP** (User Datagram Protocol) e **IP** (Internet Protocol) que son la base de la comunicación en internet hoy en día.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Application	FTP, Telnet SMTP				DNS, SNMP	
Presentation					NetBIOS	
Session						
Transport	TCP					
Network	IP	OSPF	IGRP	RIP	INT, IS-IS	ICMP
Data-Link	ARP, RARP, SNAP					
Physycal	Many Physical Implementations					

Los protocolos se pueden montar por capas, así protocolos como FTP o HTTP están una capa por encima de TCP y esta a su vez por encima de IP.

IP es la **capa de red**, encargada de crear una conexión virtual entre dos equipos enviando paquetes entre ellos ya que cada paquete IP contiene la dirección destino.

Por encima de la capa de red tenemos la **capa de transporte** donde conviven los protocolos TCP y UDP. Esta capa se encarga de refinar el direccionamiento, ya que IP sólo conecta entre equipos. TCP y UDP se encarga de conectar entre aplicaciones en una dirección dada por IP usando un puerto determinado.

TCP es un protocolo orientado a conexión lo que permite que sea fiable ante errores de forma que si la información no llega a su destino el remitente al final lo sabría. Para que haya una comunicación TCP los dos equipos deben llegar a un acuerdo de comunicación para que se establezca la conexión.



UDP es más ligero pero una vez enviado un paquete se olvida de él y no se confirma su llegada. Al contrario que TCP, no es fiable. Es no orientado a conexión. Son las aplicaciones que usan este protocolo las encargadas de lidiar con todo lo

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM
	MÓDULO	Programación de servicios y procesos		
	PROTOCOLO:	Apuntes clases	AVAL:	DATA:
	UNIDAD COMPETENCIA			
			CURSO:	2º

que UDP no hace: confirmaciones, reenvíos, ...

El uso de UDP puede parecer más limitado, pero sin embargo gracias a su ligereza está ampliamente usado hoy en día en streaming de audio y video en tiempo real donde si llega un paquete mal, no tiene sentido parar la secuencia a la espera de que llegue de nuevo el paquete. Otros usos son tareas de control en red.

Una dirección completa apunta a un equipo y un proceso en ejecución de ese equipo. El equipo lo identificamos por su **dirección IP** (4 octetos en IPv4). La aplicación por el **puerto** que es un número entre 0 y 65535. En el rango de 0-1023 están los "puertos bien conocidos o well-Known ports que están reservados para servicios estándar como FTP, POP o HTTP. Por supuesto podría darse otro uso pero no se recomienda. Un ejemplo de dirección completa sería 192.168.1.10:3240.

Suele ser un buen recordatorio usar la metáfora del correo postal:

- **IP** es como la **dirección** en un sobre (Calle, Ciudad, CP). Te dice a qué edificio (equipo) debe llegar.
- **Puerto** TCP/UDP es como el nombre del departamento o apartado dentro del edificio (Contabilidad, Pedro Martínez, Apt. 3B). Te dice a qué proceso dentro del equipo.
- **TCP** es como enviar una carta certificada con acuse de recibo. Es fiable, ordenado, pero con más trámites.
- **UDP** es como enviar una postal. Es rápida, barata, la tiras al buzón y esperas lo mejor. No sabes si llega.

Como las direcciones son complicadas de recordar para los humanos a los equipos se les puede dar un **nombre de dominio** dentro de una red. De esta forma un ordenador puede comunicarse con otro usando dicho nombre "humanizado" a través de un servidor **DNS** (Domain Name System) que se encarga de traducir su nombre a una IP. En local puede haber un emulador de este servicio mediante el archivo *hosts*.

Un ejemplo fundamental de protocolos para un caso de apocalipsis es el de IP sobre palomas mensajeras: https://es.wikipedia.org/wiki/IP_sobre_palomas_mensajeras

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Sockets

Un **socket** es una abstracción que nos permite unificar en un único elemento de programación (objeto o estructura, dependiendo del lenguaje de programación usado) un canal de comunicación con la red. Se puede ver (y se usa de forma similar) a un objeto o variable tipo archivo en un lenguaje de programación.

De esta forma cuando se inicia un socket hay que indicar datos como la dirección IP, el puerto, protocolos usados, etc. ya que es la vía de comunicación de una aplicación con la red.

Los sockets más usados son los Stream sockets que usan protocolo TCP (y son los que más vamos a usar) y los Datagram sockets que usan el protocolo UDP.

Una vez establecida la comunicación el **socket** identifica un par único

ip:puerto local + ip:puerto remoto.

Arquitectura Cliente/Servidor

Un **cliente** es una aplicación en un equipo que necesita algo y por tanto debe iniciar la comunicación con un **servidor** que es otra aplicación en el mismo u otro equipo que puede darle lo que necesita el cliente. El servidor no inicia la comunicación pero está permanentemente a la escucha por si algún cliente le requiere.

Para poder atender a múltiples clientes simultáneamente sin bloquear la escucha de nuevas conexiones, el servidor suele crear un hilo (o usar técnicas asíncronas) para atender a cada cliente aceptado, mientras el socket principal vuelve a escuchar. **Cada par IP_cliente:puerto_cliente+IP_servidor:puerto_servidor establece un canal de conexión único.**

El cliente por tanto debe conocer la IP del servidor (o la URL y usar DNS) y el puerto de la aplicación que le va a dar el servicio. Es por esto que hay una serie de puertos estándar (ftp 21, http 80, ...) para facilitar la comunicación, pero un servidor se puede colocar en el puerto que desee.

En este enlace se pueden ver la numeración de **puertos estándar**:

http://es.wikipedia.org/wiki/Anexo:Números_de_puerto

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Puedes ver los puertos abiertos en tu equipo y el proceso asociado (su PID) mediante el siguiente comando:

```
netstat -aob
```

a: muestra todas las conexiones y puertos.

o: Indica el PID del proceso asociado a la conexión

b: Indica el nombre del proceso (requiere permisos de administrador, puedes no usarla).

La clase Socket

Será la clase que representa la conexión. Permite muchas combinaciones de parámetros pero nosotros no profundizaremos demasiado en este primer acercamiento.

Nos quedaremos sólo con los elementos más estándar e imprescindibles para realizar una conexión TCP en una red IPv4 y a medida que necesitemos otras posibilidades las iremos viendo. Usaremos el siguiente constructor:

```
Socket(AddressFamily familia, SocketType tipo, ProtocolType protocolo)
```

familia: Enumerado con el que especificamos el tipo de direccionamiento usado. No se limita a IPv4 o IPv6 si no que admite otros estándares como AppleTalk, Netbios, OSI, etc...). Usaremos **InternetWork** para **IPv4**. Digamos que describe la red con la que vamos a trabajar. Por ejemplo para IPv6 usaríamos `AddressFamily.InterNetworkV6`.

tipo: El tipo de socket que vamos a usar. En principio usaremos sólo **Stream** que sirve para **conexiones bidireccionales basadas en conexión** (como TCP, pero no es la única). Pero admite otros tipos como sin conexión para UDP (Dgram) o incluso bajar a la capa de transporte (RAW) para usar otros protocolos como ICMP (El que usa el ping).

protocolo: El protocolo de comunicación usado. Usaremos normalmente **Tcp**.

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Por lo tanto el trabajo habitual con sockets para nosotros en general comenzará con este constructor:

```
Socket s = new Socket(AddressFamily.InterNetwork,
                      SocketType.Stream,
                      ProtocolType.Tcp);
```

Trabajaremos en una red TCP/IP orientada a conexión y protocolo TCP.

Una vez finalizado el uso del socket, al igual que sucede con archivos hay que **cerrarlos** mediante un **Close** o envolviéndolos en cláusulas **using** para no ocupar recursos.

Las excepciones del tipo **SocketException** son muy genéricas, pero vienen acompañadas de la propiedad **ErrorCode** que nos da más información yendo a las tablas de MSDN:

[http://msdn.microsoft.com/en-us/library/windows/desktop/ms740668\(v=vs.85\).aspx](http://msdn.microsoft.com/en-us/library/windows/desktop/ms740668(v=vs.85).aspx)

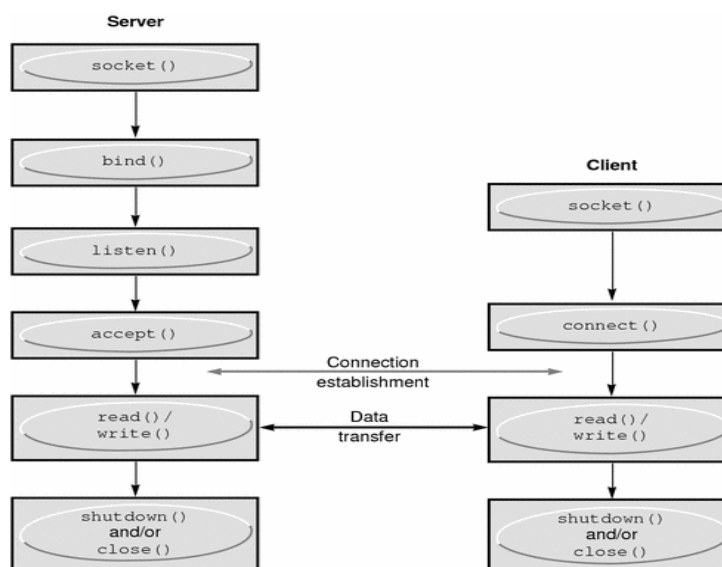
COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

Programación con sockets TCP

Como ya se comentó, un socket es una vía de conexión a la red al cual hay que indicarle varios parámetros según la forma y el destino al que nos conectaremos.

Se puede ver como una variable o clase tipo fichero (file descriptor) un tanto especial ya que en UNIX, padre del tratamiento de sockets, todos los dispositivos se tratan como archivos.

Si estamos usando un protocolo **TCP orientado a conexión**, la forma de trabajo en una aplicación cliente servidor es la siguiente:



En el **servidor** se **abre el socket**, que es la vía de comunicación, y se indica con qué protocolo se va a trabajar (en este caso sería TCP).

Mediante **bind()** se **enlaza el socket a un puerto** y una (o varias) IP de escucha.

A continuación el servidor está preparado y se pone a la **escucha con listen()**. Esta se queda a la espera de que un cliente quiera realizar una conexión.

Este es el denominado socket de escucha del servidor que siempre está en el mismo puerto y es conocido por los clientes que deseen conectarse a él.

En el **cliente** se ha **creado un socket** y a continuación lanza un **connect()** que

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

realiza una petición a la IP y al puerto donde está el servidor.

El **servidor** si lo considera apropiado (permisos, recursos, etc...) acepta la petición mediante **accept()** y la **conexión está establecida**.

En este punto si se desea que el servidor pueda atender a muchos clientes se podría crear un hilo por cada uno de los clientes tratados.

En este caso, el cliente puede usar siempre la misma dirección IP+puerto, aunque por cada una de las conexiones el sistema operativo la tratará con un socket distinto ya que consta de un par ip+puerto local e ip+puerto remoto y un protocolo determinado.

De esta forma puede usarse múltiples veces el puerto de escucha que está asociado a distintos sockets, que son los verdaderos canales de comunicación.

De forma más técnica: El sistema operativo identifica una conexión TCP única por la cuaterna: <IP_origen, Puerto_origen, IP_destino, Puerto_destino>. El socket de escucha solo usa <IP_servidor, Puerto_servidor, *, *>. Cada Accept() devuelve un **nuevo socket** con la cuaterna completa, liberando al socket principal para seguir escuchando.

Si se quiere profundizar en esto se pueden ver las respuestas en este enlace. Sobre todo son interesantes la segunda y tercera:

<https://stackoverflow.com/questions/3329641/how-do-multiple-clients-connect-simultaneously-to-one-port-say-80-on-a-server>

La comunicación se establece con comandos de envío write() o send() y de recepción de datos read() o receive() en el hilo de atención al cliente.

Usaremos funciones de más alto nivel que **send** y **receive** por comodidad: usaremos **StreamReader** y **StreamWriter** que ya los conocemos de Archivos.

Finalmente se cierra el socket cuando la comunicación ha finalizado.

En el caso de usar un protocolo UDP no orientado a conexión, no existe paso de escucha ni aceptación en el servidor, simplemente se envían y reciben los mensajes. Por el momento no lo veremos ya que la mayoría de nuestras conexiones serán TCP.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Creación de una aplicación cliente-servidor

En este caso realizaremos como ejemplo explicativo un servidor de Echo, es decir, un servidor que está a la espera de recibir un mensaje de texto de algún cliente y cuando lo recibe se lo devuelve al cliente.

Comprobación de puerto de escucha libre/ocupado

Hay varias formas de verificar si un puerto TCP está disponible para escuchar.

Una de ellas es intentar conectarnos al mismo. Es decir, si puedo conectarme es que está en uso por algún servidor ya existente o algún otro motivo (por ejemplo el firewall lo tiene bloqueado, permisos,...).

Otra es intentar ponerme a la escucha en dicho puerto para usarlo como servidor (realizar un bind). Si puedo, está libre, si no, ocupado por otra aplicación que lo está usando.

Veamos un ejemplo de esta última:

```
int port = 135;
IPEndPoint ie = new IPEndPoint(IPAddress.Any, port);

//Creacion del Socket
using (Socket s = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp))
{
    try
    {
        //Enlace de socket al puerto (y en cualquier interfaz de red)
        //Salta excepción si el puerto está ocupado
        s.Bind(ie);
        Console.WriteLine($"Port {port} free");
        s.Listen(1); // Prueba mínima para asegurar que funciona como servidor
    }
    catch (SocketException e) when (e.ErrorCode ==
(int)SocketError.AddressAlreadyInUse)
    {
        // ó 10048
        Console.WriteLine($"Port {port} in use");
    }
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

    catch (SocketException e) // Error inesperado
    {
        Console.WriteLine($"Error: {e.SocketErrorCode} - {e.Message}");
    }
}

```

La clase **IPAddress** es una de las que dispone .Net para manejar direcciones IP de forma encapsulada. Tiene varias posibilidades de darle una IP determinada. La más habitual es **Parse** o **TryParse** para hacerlo a partir de un string.

Por ejemplo:

```
IPAddress ip = IPAddress.Parse("192.168.20.1");
```

Hay que tener cuidado al usarla pues realiza ciertas interpretaciones al pasarle ciertos valores que en principio no son IP's válidas. Para tener más información revisa el apartado Remarks del siguiente enlace:

<https://learn.microsoft.com/en-us/dotnet/api/system.net.ipaddress.tryparse?view=net-7.0>

Pero también tiene valores predefinidos. Por ejemplo:

- **IPAddress.Loopback** : normalmente 127.0.0.1
- **IPAddress.Any**: Referido a una IP cualquiera que escucha en todos los interfaces de red. La usaremos habitualmente en servidores. Cuando usas `IPAddress.Any`, el servidor acepta conexiones tanto desde la red local como desde internet (si el firewall lo permite).

IPEndPoint: Como se comentó previamente, para realizar una conexión no llega una dirección IP si no que se necesita un par IP+puerto (de hecho dos pares, local y remoto). Para ello .Net dispone de la clase **IPEndPoint** que encapsula estos dos datos en un único objeto.

El constructor más habitual que usaremos dispone precisamente de esos dos parámetros tal y como se ve en el ejemplo previo.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Realización de la aplicación: Lado del Servidor

Empezaremos en este apartado a realizar la parte del servidor paso a paso estableciendo el protocolo de comunicación.

En principio **lo probaremos con Telnet** y posteriormente haremos un cliente que cumpla el protocolo del servidor.

Servidor: Conexión

Los pasos son parte de los vistos en el gráfico de conexión mediante protocolo TCP:

1. Creación del socket - `new Socket()`
2. Apertura del puerto - `Bind()`
3. Escucha - `Listen()`
4. Aceptación de la petición del cliente en cuanto exista - `Accept()`

```

IPEndPoint ie = new IPEndPoint(IPAddress.Any, 31416);

// Creacion del Socket. Un socket es IDisposable,
// por tanto con using para cierre automático.
using (Socket s = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp))
{

    // Enlace de socket al puerto (y en cualquier interfaz de red)
    // Ojo: Habría que realizar comprobación de puerto ocupado
    // Según se vio previamente
    s.Bind(ie);

    // Esperando una conexión y estableciendo cola de clientes pendientes
    s.Listen(10);

    // Información de depuración
    Console.WriteLine($"Servidor iniciado. Escuchando en {ie.Address}:{ie.Port}");
    Console.WriteLine("Esperando conexiones... (Ctrl+C para salir)");

```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```
// Esperamos y aceptamos la conexión del cliente (socket bloqueante)
// Luego se hará en bucle para aceptar varios clientes
Socket sClient = s.Accept();

using (sClient) // Este using por separado pues luego irá en un hilo
{
    // Obtenemos la info del cliente. El casting es necesario ya que
    // RemoteEndPoint es del tipo EndPoint mas genérico
    IPEndPoint ieClient = (IPEndPoint)sClient.RemoteEndPoint;
    Console.WriteLine($"Cliente conectado:{ieClient.Address} en puerto
{ieClient.Port}");

    // Aquí se cierra la conexión,
    // luego se atenderá al cliente en un hilo
}
}
```

Notas sobre el código anterior:

- El parámetro **backlog** define el tamaño de la cola de conexiones pendientes (clientes que han realizado *Connect()* pero aún no han sido procesados por *Accept()*). No es el número máximo de clientes simultáneos que el servidor puede manejar. Por ejemplo, con *backlog* = 3, si cuatro clientes intentan conectarse casi simultáneamente, los primeros tres entrarán en la cola y el cuarto recibirá un error de conexión rechazada..
- Se debería comprobar si el puerto que vamos a usar está en uso mediante un control de excepciones según se vio previamente.
- Se trabaja con dos sockets, el del servidor que es el primero que se crea (*s: socket de escucha*) y el del cliente en cuanto llega uno (*sCliente*). Más adelante veremos cómo tratar varios clientes en varios hilos.

Es importante recalcarlo: *Accept()* devuelve un nuevo socket (*sClient*) dedicado exclusivamente a la comunicación con ese cliente. El socket original (*s*) queda libre para seguir aceptando nuevas conexiones. Esto permite atender múltiples clientes simultáneamente.

- Puedes probar el servidor anterior ejecutando desde consola:

```
telnet 127.0.0.1 31416
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

En versiones de Windows Vista en adelante no viene instalado el cliente telnet. Para instalarlo ve a panel de control → Programas y características → Activar y desactivas características de Windows.

Ahí puedes seleccionar cliente Telnet.

Otra opción es usar otro programa tipo telnet como putty o el Ubuntu integrado que se puede descargar de la MS Store y activando el subsistema para Linux (en el mismo sitio donde activas el telnet).

Si ejecutas un netstat antes del cierre de los sockets en el servidor puedes ver algo similar a esto:

```
C:\>netstat -ao
```

Conexiones activas

Proto	Dirección local	Dirección remota	Estado
...			
TCP	0.0.0.0:31416	A22-PC32:0	LISTENING
...			
TCP	127.0.0.1:31416	A22-PC32:50318	ESTABLISHED

La primera línea es el socket de escucha y la segunda el de cliente.

Interpretación del netstat:

- `0.0.0.0:31416` → Socket de escucha (acepta conexiones de cualquier IP)
- `LISTENING` → Estado de espera de conexiones
- `127.0.0.1:31416` → Conexión establecida desde localhost
- `50318` → Puerto efímero asignado al cliente por el SO
- `ESTABLISHED` → Conexión activa entre cliente y servidor

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Servidor: transmisión y recepción simple

Una vez realizada la conexión, el servidor le envía al cliente un mensaje de bienvenida (estamos estableciendo un protocolo de comunicación) que el cliente tendrá que recibir. A continuación el cliente envía también un mensaje que el servidor recibirá y mostrará por pantalla.

Esto se puede hacer a bajo nivel enviando byte a byte como se puede ver en el ejemplo del [Apéndice I](#). Pero nosotros por simplicidad vamos a usar una capa intermedia de Streams que ya los conocemos de usarlos con archivos. Es decir, los solucionaremos usando **StreamReader** y **StreamWriter**.

Además esta es una solución existente **en** otros lenguajes como **Java**, no siendo exclusivo de .Net y por tanto no perderemos generalidad:

https://poliformat.upv.es/access/content/group/OCW_6069_2008/T2.-Comunicación%20I%3A%20del%20C_S%20al%20modelo%20de%20objetos/Tecnolog%C3%ADa%20JAVA/Java-sockets.pdf

Estas dos clases sabemos como usarlas para manejar archivos de texto. Pero de forma genérica se le puede pasar un stream de bytes cualesquiera. De esta forma usaremos un stream de red (**NetworkStream**) para inicializar estas conocidas clases en lugar de un archivo del disco duro o un file stream y luego usaremos WriteLine y ReadLine para enviar y recibir mensajes de texto respectivamente.

La clase NetworkStream es un Stream especializado que encapsula un socket, permitiendo leer/escribir bytes a través de la red. Al igual que FileStream nos permite acceder a archivos, NetworkStream nos permite acceder a conexiones de red. Esta abstracción es poderosa porque podemos usar las mismas clases (StreamReader, StreamWriter, BinaryReader, etc.) que ya conocemos.

Socket → NetworkStream → StreamReader/StreamWriter

↓
Bytes

↓
Bytes

↓
Texto (string)
(por defecto con codificación UTF-8)

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

Dentro del bloque using (sClient), al final, añade el siguiente código:

```
// Creación del Stream de Red.
// que StreamReader y StreamWriter aceptan
// como parámetro en el constructor
using (NetworkStream ns = new NetworkStream(sClient))
using (StreamReader sr = new StreamReader(ns))
using (StreamWriter sw = new StreamWriter(ns))
{
    string welcome = "Welcome to The Echo-Logic, Odd, Desiderable, " +
        "Incredible, and Javaless Echo Server (T.E.L.O.D.I.J.E Server)";

    // El envío por red se convierte en un WriteLine
    sw.WriteLine(welcome);

    // Con flush se fuerza el envío de los datos sin esperar al cierre
    // Otra opción es: sw.AutoFlush = true
    sw.Flush();

    // Esperamos a la recepción del mensaje
    // El ReadLine es bloqueante.
    // Más adelante veremos comprobaciones sobre lo recibido.
    string msg = sr.ReadLine();

    // Se muestra por consola lo que manda el cliente
    if ( msg != null)
    {
        Console.WriteLine(msg);
    }
    else
    {
        Console.WriteLine("El cliente cerró la conexión.");
    }
    // Finalización de la conexión
}
```

Primero se crea un NetworkStream que haga de puente entre el socket y los StreamWriter y StreamReader que ya conocemos del tema de archivos.

Para enviar un mensaje por red el servidor realiza una operación de escritura sobre el stream correspondiente.

Como normalmente las escrituras sobre Streams son en memoria, como ocurría con archivos, y los datos no se vuelcan al destino final hasta que se cierran dichos streams, es necesario hacer un **flush** para realizar el envío de datos cada vez que

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM	
	MÓDULO	Programación de servicios y procesos			CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:
	UNIDAD COMPETENCIA				

se hace un Write. La recomendación con sockets es poner la propiedad AutoFlush del StreamWriter a true, de esta forma puedes obviar lo Flush() pues se hacen automáticamente.

A continuación el servidor se queda esperando a que llegue un mensaje del cliente haciendo un ReadLine. Ten en cuenta que esta línea es bloqueante.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Servidor: intercambio de datos

Finalmente el intercambio de mensajes en este caso se transforma en un bucle en el que el servidor espera a recibir un mensaje con un ReadLine y cuando lo recibe lo devuelve con un WriteLine sobre los streams correspondientes.

Se presenta el código desde la creación de los streams, sustituye al anterior:

```
// Indicamos mediante sobrecarga la codificación de la consola.
// Por defecto c# trabaja con Encoding.UTF8
Encoding codificacion = Console.OutputEncoding;
using (NetworkStream ns = new NetworkStream(sClient))
using (StreamReader sr = new StreamReader(ns, codificacion))
using (StreamWriter sw = new StreamWriter(ns, codificacion))
{
    sw.AutoFlush = true; // Flush automático.
    string welcome = "Welcome to The Echo-Logic, Odd, Desiderable, " +
        "Incredible, and Javaless Echo Server (T.E.L.O.D.I.J.E Server)";
    sw.WriteLine(welcome);

    string? msg = "";

    while (msg != null)
    {
        try
        {
            // Leemos el mensaje del cliente
            msg = sr.ReadLine();

            // Si el cliente manda mensaje se le envía de vuelta
            // pero si manda null es que ha desconectado.
            if (msg != null)
            {
                Console.WriteLine($"El cliente dice {msg}");
                sw.WriteLine($"El servidor dice {msg}");
            }

            // Si se cierra el cliente, salta excepción
            // Al siguiente readline que será null
            catch (IOException)
            {
                msg = null;
            }
        }
        Console.WriteLine("Cliente desconectado.\nConexión cerrada");
    }
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

La forma en que se cierra la conexión en este caso es exclusivamente que el cliente se cierre abruptamente (cierra la consola del telnet para probarlo con el icono X). En este caso, si el cliente no empezó a escribir nada el ReadLine devuelve null y a la siguiente vuelta del bucle, termina.

Puedes probar a quitar el try/catch y en principio parece que funciona igual, sin embargo, puede darse el caso que el cliente empiece a escribir algo pero le da al icono X (cierre) antes de que se envíe ese mensaje (retorno de carro). En ese caso el mensaje no es null y al hacer el luego el *WriteLine* saltaría excepción, de ahí la necesidad del try.

De esta forma tenemos nuestro servidor finalizado.

Es importante indicar que **estamos usando sockets con bloqueo** de forma que hay que alternar entre los WriteLine y ReadLine de cliente y servidor para que no entren ninguno de los programas (ni cliente ni servidor) en deadlock. Existe la posibilidad de usar sockets de forma asíncrona de forma que el programa no se bloquee. Existen varias posibilidades; nosotros usaremos hilos para solventarlo más adelante.

Si quieres poner un tiempo de espera limitado para el cliente, se puede establecer un timeout. Por ejemplo si añades la siguiente línea:

```
sr.BaseStream.ReadTimeout = 2000; //milliseconds, son 2 seconds
```

tras la creación del StreamReader (por ejemplo antes del mensaje de bienvenida), se provoca que si el cliente tarda más de 2 segundos en enviar algún mensaje entonces se produce la desconexión (también salta excepción).

Finalmente comentar que la comunicación entre máquinas diferentes suele hacerse en UTF8, aquí la cambiamos para evitar los errores de codificación de la consola de Windows. También es práctica habitual **negociar la codificación** al principio de la comunicación.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Servidor multihilo

Existen diversas formas de hacer que un socket no bloquee el programa en ejecución a la espera de realizar alguna acción. En este apartado veremos una de ellas que consiste en lanzar las operaciones que pueden bloquear el programa en un hilo aparte.

De esta forma al no estar el programa bloqueado, se pueden atender a varios clientes simultáneamente, cosa que no hemos hecho por el momento.

En el servidor el socket es común. Sólo hay un puerto de espera. Una vez aceptada la conexión con el cliente es cuando se lanza el hilo.

El algoritmo de esto se presenta a continuación.

El **bucle principal** de tratamiento de clientes:

```

creación del socket
bind
listen
while (servidor activo)
    acepta conexión
    lanza hilo con ese cliente
fin while

```

En el **hilo de cada cliente** el resto de las tareas:

```

envío de mensaje de bienvenida
bucle de envío y recepción de datos (según protocolo)
cierre del socket

```

Cuando el servidor acepta la conexión, obtiene un socket de cliente que le devuelve el método Accept. Este socket es el objeto parámetro que hay que pasar al thread (En ciertas situaciones llega con pasar el StreamWriter en vez del socket).

Tomando como referencia el servidor visto anteriormente, lo realizaremos en versión multihilo y con una estructura más orientada a objetos.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Crearemos la **clase EchoServer** donde habrá dos funciones: **InitServer** donde estará el bucle principal que está a la esucha y la función **ClientDispatcher** que es la que se lanzará en hilo por cada cliente y donde estará el protocolo.

La clase sería:

```
public class EchoServer
{
    // Se usaría para parar el servidor
    public bool ServerRunning { set; get; } = true;

    public int Port { get; set; } = 31416;

    public void InitServer()
    {
        IPEndPoint ie = new IPEndPoint(IPAddress.Any, Port);

        using (Socket s = new Socket(AddressFamily.InterNetwork,
                                     SocketType.Stream,
                                     ProtocolType.Tcp))
        {
            s.Bind(ie);
            s.Listen(10);

            Console.WriteLine($"Servidor iniciado. " +
                              $"Escuchando en {ie.Address}:{ie.Port}");
            Console.WriteLine("Esperando conexiones... (Ctrl+C para salir)");

            // Bucle para atender clientes de forma continua.
            while (ServerRunning)
            {
                Socket client = s.Accept();

                // Se crea hilo para atender el nuevo cliente.
                Thread hilo = new Thread(() => ClientDispatcher(client));

                // Aquí se podría configurar la prioridad y
                // background si fuera necesario

                // Se lanza el hilo y se espera a un nuevo cliente
                hilo.Start();
            }
        }
    }
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

private void ClientDispatcher(Socket sClient)
{
    using (sClient)
    {
        IPEndPoint ieClient = (IPEndPoint)sClient.RemoteEndPoint;
        Console.WriteLine($"Cliente conectado:{ieClient.Address} " +
            $"en puerto {ieClient.Port}");

        Encoding codificacion = Console.OutputEncoding;
        using (NetworkStream ns = new NetworkStream(sClient))
        using (StreamReader sr = new StreamReader(ns, codificacion))
        using (StreamWriter sw = new StreamWriter(ns, codificacion))
        {
            sw.AutoFlush = true;
            string welcome = "Welcome to The Echo-Logic, Odd, Desiderable, " +
                "Incredible, and Javaless Echo Server (T.E.L.O.D.I.J.E Server)";
            sw.WriteLine(welcome);

            string? msg = "";

            while (msg != null)
            {
                try
                {
                    msg = sr.ReadLine();

                    if (msg != null)
                    {
                        Console.WriteLine($"El cliente dice {msg}");
                        sw.WriteLine($"El servidor dice {msg}");
                    }
                }
                catch (IOException)
                {
                    msg = null;
                }
            }
            Console.WriteLine("Cliente desconectado.\nConexión cerrada");
        }
    }
}

```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Para lanzar el servidor, por ejemplo desde un Main simplemente se instancia y se ejecuta la función de inicio:

```
internal class Program
{
    static void Main(string[] args)
    {
        (new EchoServer()).InitServer();
    }
}
```

Con lo que se ha visto de sockets y de hilos debería entenderse el funcionamiento del código anterior.

Se puede pensar que al haber varios clientes conectados al mismo puerto del servidor puede haber algún conflicto, pero no es así, recordemos que cada conexión se identifica por los dos pares de IP+puerto tanto del cliente como del servidor (Además del protocolo).

Existen otras formas de tratar el bloqueo de los sockets pero no los veremos por el momento.

Algunas cuestiones a tener en cuenta:

- Al no limitar la cantidad de clientes podría hacerse un ataque de denegación de servicio. Esto se impediría limitando el n.º de hilos en ejecución.
- Si se quiere parar el servidor, es necesario además de cambiar la booleana ServerRunning, cerrar el socket de escucha. Esto provoca una excepción que se controlaría con try-catch. Por ejemplo podría hacerse la función siguiente:

```
// El socket de escucha pasa a ser campo privado
// de instancia para usarlo tanto en InitServer como en StopServer
private Socket s;
public void StopServer()
{
    Console.WriteLine("Deteniendo servidor");
    ServerRunning = false;
    s.Close();
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

Y luego en InitServer, el bucle quedaría:

```
try
{
    while (ServerRunning)
    {
        Socket client = s.Accept();// Salta excepción si se cierra s.
        Thread hilo = new Thread(() => ClientDispatcher(client));
        hilo.Start();
    }
}
catch (SocketException)
{
    Console.WriteLine("Fin de servidor");
}
```

- Otra forma de evitar Accept bloqueante es usar tareas con AcceptAsync.

Como introducción a los servidores con hilos esta es una solución correcta pero para un servidor más eficiente convendría usar Task tal y como se ve en el [Apéndice III](#).

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Cliente de eco gráfico

Como ejemplo de cliente, se usará Window Forms y await/Task para evitar que el IU se bloquee.

Crea un proyecto Windows Forms y añade un textbox (**txtEnviar**) y un botón (**btnEnviar**).

Añade un segundo textbox (**txtRecibir**) pero este que sea **multilinea**.

La idea es que el usuario escriba lo que quiera en txtEnviar, y al pulsar el botón (se recomienda configurarlo como AccpetButton) se envíe dicho texto al servidor de Eco y lo que se reciba se muestre en txtRecibir.

Dentro de la clase Form1 añade las siguientes variables y método:

```
IPAddress ip = IPAddress.Parse("127.0.0.1");
int puerto = 31416;

private async Task<string> EnvioYRecepcionAsync()
{
    try
    {
        // Se crea el socket para conectarse al servidor
        using (Socket conexion = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp))
        {
            IPEndPoint ep = new IPEndPoint(ip, puerto);

            // Trata de conectarse a la IP y Puerto configurados.
            await conexion.ConnectAsync(ep);

            // Si hay éxito (si no salta excepción), se procede al protocolo:
            // - Recepción de mensaje de bienvenida.
            // - Envío de texto desde txtEnviar
            // - Recpción de mensaje de servidor
            Encoding codificacion = Console.OutputEncoding;
            using (NetworkStream ns = new NetworkStream(conexion))
            using (StreamReader sr = new StreamReader(ns, codificacion))
            using (StreamWriter sw = new StreamWriter(ns, codificacion))
            {
                sw.AutoFlush = true;
            }
        }
    }
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

        // Leemos mensaje de bienvenida y lo deshechamos
        // (No hacemos nada con él).
        string msg = await sr.ReadLineAsync();

        // Enviamos dato
        await sw.WriteLineAsync(txtEnviar.Text);

        // Esperamos recepción
        msg = await sr.ReadLineAsync();

        return msg;
    }
}
}
catch (Exception ex) when (ex is SocketException || ex is IOException)
{
    // Problema de comunicación:
    // no existe servidor, no responde, red caída,...
    return ex.Message;
}
catch (Exception ex)
{
    return $"Error inesperado: {ex.GetType().Name}. Contacte con soporte.";
}
}

```

A continuación crea una función enlazada al evento click del botón con el siguiente código:

```

private async void btnEnviar_Click(object sender, EventArgs e)
{
    // Desactivamos el botón hasta que se reciba el mensaje (no es imprescindible)
    btnEnviar.Enabled = false;
    string msg = await EnvioYRecepcionAsync();

    // Tras la recepción mostramos en txtRecibir el mensaje recibido.
    txtRecibir.AppendText(msg + Environment.NewLine);
    btnEnviar.Enabled = true;
}

```

También se puede **añadir en el servidor** la línea:

```
Thread.Sleep(2000);
```

Justo antes del `sw.WriteLine(msg)` para provocar que tarden las respuestas y se vea que el GUI no se bloquea.

Si cierras el servidor, el cliente informará del problema pero sin bloquearse.

COLEXIO VIVAS S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Otras clases para trabajo en Red

Hemos visto en este tema la base de las conexiones en red mediante sockets. Es cierto que cualquier tipo de intercambio de datos o protocolo se puede hacer con las herramientas vistas, sin embargo existen en .Net multitud de clases que nos facilitan en gran medida el trabajo en red dependiendo de los protocolos con los que queramos trabajar. Veamos algunos ejemplos:

TcpClient, TcpListener, UdpClient: Clases que simplifican la conexión por sockets. No son tan potentes, pero la inicialización es mucho más sencilla.

WebClient: Clase para comunicaciones HTTP.

System.Web.Mail: Espacio de nombres para la gestión de correo electrónico.

Remoting: posibilidad de tener programas distribuidos en varios equipos y que desde unos se puedan ejecutar funciones sitas en otros.

System.Security.Cryptography: Espacio de nombres para la gestión de seguridad.

NetworkInterface: Clase que nos da información sobre los distintos interfaces de red y algunos estadísticos de los mismos.

HttpListener: Clase para incluir en una aplicación la posibilidad de aceptar y responder peticiones HTTP.

En general si vas a realizar algún tipo de intercambio de información por red comprueba primero la existencia de clases para dicho intercambio lo cual te facilitará la labor en gran medida.

COLEXIO VIVAS S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Apéndice I: Cliente-servidor con send/receive

En este apéndice se verá la realización de un programa con su cliente y su servidor pero trabajando a bajo nivel, con envío de bytes usando send/receive en lugar de Streams como se vio durante el tema.

Servidor: partimos del mensaje de bienvenida, lo anterior es igual.

Una vez realizada la conexión, el servidor le envía al cliente un mensaje de bienvenida (estamos estableciendo un protocolo de comunicación) que el cliente tendrá que recibir.

El código del servidor sería:

```
string bienvenida = "Bienvenido al servidor de Echo, tu di que yo repito";
byte[] datos = Encoding.UTF8.GetBytes(bienvenida);
sCliente.Send(datos);
```

La transmisión de datos a través de sockets se hace como un stream de bytes, concretamente como un vector de bytes. Por tanto convertimos lo que queramos mandar a un vector de bytes mediante la clase Encoding (se encuentra en el namespace *System.Text*).

Esta clase tiene varias posibilidades. Usamos UTF8 para poder enviar distintos juegos de caracteres.

Al método Send se le pasa el vector de bytes que lo envía al cliente, este último debe tener preparado un Receive para obtener estos datos.

Nuevamente puedes probarlo con el telnet.

Cliente: recepción

A continuación el código del cliente que cumple el protocolo anterior y recibe el mensaje de bienvenida:

```
byte[] datos=new byte[512];
int nDatos = servidor.Receive(datos);
string mensaje = Encoding.UTF8.GetString(datos, 0, nDatos);
Console.WriteLine(mensaje);
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Se debe establecer el tamaño máximo de mensaje que se va a enviar (esto debe ser parte del protocolo).

El método Receive devuelve el número de bytes recibidos y en el parámetro coloca los datos recibidos.

Cuando el cliente remoto cierra una sesión el método Receive devuelve 0. Si no devuelve la cantidad de bytes recibidos.

Encoding.UTF8.GetString vuelve a convertir a cadena los bytes que previamente convertimos a UTF8.

Servidor: intercambio de datos

Finalmente mediante un bucle hacemos funcionar el servidor de Echo:

```
while(true)
{
    datos = new byte[512];
    int nDatos = sCliente.Receive(datos);
    if (nDatos == 0)
        break;
    string mensaje=Encoding.UTF8.GetString(datos, 0, nDatos);
    Console.WriteLine(mensaje);
    sCliente.Send(datos, nDatos, SocketFlags.None);
}
```

Puedes probarlo mediante telnet. Verás que lo que se escribe desde la consola lo devuelve repetido. Puedes tener problemas con la codificación: prueba ASCII en lugar de UTF8.

En cada vuelta del bucle se inicializa el vector de datos. Luego se espera que el cliente envíe datos. Si hay datos recibidos (Receive devuelve una cantidad mayor que 0) se transforman en string y se devuelven al cliente mediante Send. Previamente se muestran por pantalla.

Es necesario usar una sobrecarga distinta del Send en la cual se indica la cantidad de datos real que se quiere enviar porque si no se envían los 512 bytes y sólo nos

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

interesa la parte del mensaje. Del enumerado SocketFlag no es necesario preocuparse por el momento, simplemente se indica que no hace falta ningún flag (no es necesario cambiar el comportamiento del socket) mediante None.

Cliente: Intercambio de datos

En este caso el bucle de intercambio para cumplir el protocolo es:

```
while (true)
{
    mensaje = Console.ReadLine();
    if (mensaje == "salir")
        break;
    servidor.Send(Encoding.UTF8.GetBytes(mensaje));
    datos = new byte[1024];
    nDatos = servidor.Receive(datos);
    mensaje = Encoding.UTF8.GetString(datos, 0, nDatos);
    Console.WriteLine(mensaje);
}
//Finalizamos la conexión.
//Esto provoca que Receive en el servidor devuelve 0
Console.WriteLine("Terminando la conexión...");
servidor.Shutdown(SocketShutdown.Both);
```

En este bucle se pide una cadena al usuario. Con la palabra *salir* se finaliza la conexión. En caso contrario se le envía dicho mensaje al servidor y se espera que nos devuelva dicho mensaje si todo funciona correctamente. Finalmente se muestra.

El método Shutdown es necesario para indicar que se finalizan las transmisiones y es el que provoca que el Receive en el servidor devuelva 0 para detectar dicho fin de transmisión.

Otras consideraciones:

- Si se usa un buffer de recepción muy pequeño, en sucesivos envíos nos irá llegando los datos anteriores que no se han metido en el buffer (realmente se encuentran en el buffer TCP interno del sistema). Prueba metiendo en el cliente un buffer (vector *datos*) de tamaño 10 por ejemplo.
- Nada asegura que un Receive vaya a recibir datos de un único Send, puede darse el caso de que si se dan varios Send seguidos, al hacer un Receive se

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

cojan los datos de dos o más Sends si ya están en el buffer del sistema (en el siguiente apartado vemos el problema y una solución).

- Como solución a los puntos anteriores hay varias posibilidades: mensajes siempre del mismo tamaño, enviar el tamaño con el mensaje (se ve más abajo) o usar una marca para separar mensajes. También podemos usarlo cómo veremos más adelante con Streams.
- Si se usa la propiedad ReceiveTimeout de un socket a cierta cantidad de ms, una vez ejecutado el Receive si pasan los ms indicados salta una excepción.
- Estamos usando sockets con bloqueo de forma que hay que alternar entre los send y receive para que no entren ninguno de los programas (ni cliente ni servidor) en deadlock. Existe la posibilidad de usar sockets de forma asíncrona de forma que el programa no se bloquee. Existen varias posibilidades; nosotros usaremos hilos para solventarlo.
- No veremos UDP en este tema, pero las diferencia básicas son la creación del socket y el envío y recepción de los datos. El socket se crearía de la siguiente forma:

```
Socket s = Socket(AddressFamily.InterNetwork, SocketType.Dgram,
                  ProtocolType.Udp);
```

Y para enviar y recibir se usarían respectivamente los métodos *SendTo()* y *ReceiveFrom()*.

Cómo se comentó anteriormente, puede haber un problema a la hora de enviar y recibir mensajes ya que un Receive no tiene por que se equivalente a un Send, si no que recoge lo que tiene disponible en el buffer TCP del sistema.

Veamos un ejemplo. Usamos el siguiente cliente:

```
#if LOCAL
    const string IP_SERVIDOR = "127.0.0.1";
#else
    const string IP_SERVIDOR = "10.211.55.2";
#endif

IPEndPoint ie=new IPEndPoint(IPAddress.Parse(IP_SERVIDOR), 31416);
Console.WriteLine(
    "Iniciando cliente. Pulsa una tecla para iniciar la conexión");
Console.ReadKey();
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

```

Socket servidor = new Socket(AddressFamily.InterNetwork,
                             SocketType.Stream, ProtocolType.Tcp);
try
{
    servidor.Connect(ie);
}
catch (SocketException e)
{
    Console.WriteLine(
        "Error de conexión: {0}\nCódigo de error: {1}({2})" ,
        e.Message, (SocketError)e.ErrorCode, e.ErrorCode);
    Console.ReadKey();
    return;
}

byte[] datos=new byte[1024];
int nDatos = servidor.Receive(datos);
string mensaje = Encoding.UTF8.GetString(datos, 0, nDatos);
Console.WriteLine(mensaje);

string[] dias = new string[] { "Lunes", "Martes", "Miércoles", "Jueves",
                              "Viernes", "Sábado", "Domingo" };
foreach (string dia in dias)
{
    servidor.Send(Encoding.UTF8.GetBytes(dia));
    //System.Threading.Thread.Sleep(100);
}

Console.WriteLine("Terminando la conexión...");
servidor.Shutdown(SocketShutdown.Both);
servidor.Close();
Console.ReadKey();

```

Básicamente es el mismo cliente usado anteriormente. El cambio estriba a partir del mensaje de bienvenida. En este caso enviamos una serie de mensajes todos seguidos.

El servidor (receptor) de los mensajes puede ser el mismo. Puedes comentar la línea

```
sCliente.Send(datos, nDatos, SocketFlags.None);
```

que provoca una excepción.

Observarás que al recibir la información pone todos (o algunos, dependiendo del estado de la red) los mensajes en la misma línea. Si quitas el comentario a la línea

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

del Sleep sin embargo se recibe cada mensaje en una línea que es como debe ser.

Esto es porque llegan varios mensajes al buffer TCP del sistema antes de ejecutar el Receive, y cuando este se ejecuta se leen todos a la vez.

Para solucionar esto existen varias posibilidades, algunas ya comentadas:

- Mandar mensajes del mismo tamaño
- Mandar como integer antes del mensaje el tamaño en bytes del mismo
- Usar un carácter especial como marca divisoria de mensajes
- Usar Streams similares a los de archivos aplicados a la red.

A continuación se puede ver la solución mediante el envío del tamaño del mensaje. Es una solución simple a partir de lo que ya conocemos de sockets por lo que no debería tener mayor dificultad su comprensión.

Envío de mensajes con su tamaño

Esta solución se implementa con una función especialmente creada para enviar y otra para recibir que ejecutan de forma más completa el Send y el Receive respectivamente teniendo en cuenta el problema visto anteriormente.

Otro problema que tiene en cuenta es que el método Send no tiene por que enviar todos los datos de un mensaje. De hecho este método devuelve la cantidad de datos que ha enviado de forma que podemos saber los que quedan pendientes por enviar.

```
public static int envioMensaje(Socket s, byte[] mensaje)
{
    int enviados;
    byte[] tam = BitConverter.GetBytes(mensaje.Length);
    int total = 0;
    int pendientes = mensaje.Length;

    //Enviamos el tamaño del mensaje
```

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

```

    enviados = s.Send(tam);

    //Y luego los datos
    while (total < mensaje.Length)
    {
        //Si se envía el mensaje en la primera vuelta, todo OK.
        //Si no en sucesivas vueltas se envía el resto del mensaje que no se
    envió

        enviados = s.Send(mensaje, total, pendientes, SocketFlags.None);
        total += enviados;
        pendientes -= enviados;
    }
    return total;
}

public static byte[] recibeMensaje(Socket s)
{
    int total = 0;
    int recv;
    byte[] tam = new byte[4];

    //Recibimos el tamaño del mensaje
    recv = s.Receive(tam, 0, 4, 0);

    //Finalizamos si Receive nos devuelve 0
    if (recv == 0)
        return Encoding.UTF8.GetBytes("salir");

    int tamInt = BitConverter.ToInt32(tam, 0);
    int pendientes = tamInt;
    byte[] mensaje = new byte[tamInt];

    //Recibimos datos parando cuando se haya completado el tamaño
    while (total < tamInt)
    {
        recv = s.Receive(mensaje, total, pendientes, 0);
        total += recv;
        pendientes -= recv;
    }
    return mensaje;
}

```

Parte del cliente que cambia (Las funciones anteriores se supone que están en el Servidor, por lo que hay que añadir la referencia al Cliente para poder usarlo):

(...)

```

string mensaje = Encoding.UTF8.GetString(Servidor.Program.recibeMensaje(servidor));
Console.WriteLine(mensaje);

```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

string[] dias = new string[] { "Lunes", "Martes", "Miércoles", "Jueves", "Viernes",
                                "Sábado", "Domingo" };
foreach (string dia in dias)
{
    Servidor.Program.envioMensaje(servidor, Encoding.UTF8.GetBytes(dia));
}
(...)

```

Parte del servidor que cambia:

(...)

```

string bienvenida = "Bienvenido al servidor\n";
byte[] datos = Encoding.UTF8.GetBytes(bienvenida);
envioMensaje(sCliente, datos);

for ( ; ; )
{
    string mensaje = Encoding.UTF8.GetString(recibeMensaje(sCliente));
    if (mensaje == "salir")
        break;
    Console.WriteLine(mensaje);
    System.Threading.Thread.Sleep(500);
}
(...)

```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2°
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

Apéndice II: Información de los interfaces de red mediante el registro de Windows

Además de obtener información lógica sobre la red, en ocasiones es necesario disponer de más información sobre el hardware utilizado. Esto se puede hacer mediante la clase **NetworkInterface** pero también accediendo directamente al registro de Windows a partir del WMI (Windows Management Instrumentation) que es una forma estándar de acceder a la información del sistema y que vamos a ver aquí como ejemplo. Se puede ver como una manera de acceder al registro de Windows viéndolo como una base de datos a través de sentencias SQL.

Al ser información propia de Windows no son clases que encontremos en el framework de Mono ya que este último es independiente de la arquitectura.

A continuación un ejemplo de este segundo caso. **Para que funcione** es necesario agregar System.Management a las referencias justo debajo del proyecto en el explorador de soluciones:

```
using System;
using System.Management;

class WMICardGrab
{
    public static void Main()
    {
        ManagementObjectSearcher query = new
        ManagementObjectSearcher("SELECT * FROM Win32_NetworkAdapterConfiguration
WHERE IPEnabled = 'TRUE'");
        ManagementObjectCollection queryCollection = query.Get();
        foreach (ManagementObject mo in queryCollection)
        {
            string[] addresses = (string[])mo["IPAddress"];
            string[] subnets = (string[])mo["IPSubnet"];
            string[] defaultgateways =
                (string[])mo["DefaultIPGateway"];
            Console.WriteLine("Network Card: {0}", mo["Description"]);
            Console.WriteLine("  MAC Address: {0}", mo["MACAddress"]);
            foreach (string ipaddress in addresses)
            {
                Console.WriteLine("    IP Address: {0}", ipaddress);
            }
            foreach (string subnet in subnets)
            {
                Console.WriteLine("    Subnet Mask: {0}", subnet);
            }
        }
    }
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

    }
    foreach (string defaultgateway in defaultgateways)
    {
        Console.WriteLine(" Gateway: {0}", defaultgateway);
    }
}
Console.ReadKey();
}
}

```

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

Apéndice III: Servidor multihilo usando Task.

A continuación se presenta el mismo servidor de Eco que se hizo con la clase Thread pero usando Task/await. Con esto se logra servidores que acepten más conexiones ocupando menos recursos. El código sigue una estructura similar a la del servidor con Thread y es autoexplicativo mediante comentarios:

```
using System.Diagnostics;
using System.Net;
using System.Net.Sockets;
using System.Text;

public class EchoServerAsync
{
    // Usamos CancellationTokenSource en lugar de bool para cancelación.
    // Esto permite:
    // - Notifica a AcceptAsync() y otras operaciones pendientes.
    // - Propaga la cancelación a todas las tareas hijas del servidor.
    // - Más robusto que una simple variable booleana compartida, pues
    // garantiza que todos los hilos ven el cambio inmediatamente.

    private CancellationTokenSource? cancellationTokenSource;

    public int Port { get; set; } = 31416;

    // Método pasa a ser asíncrono
    public async Task InitServerAsync()
    {
        cancellationTokenSource = new CancellationTokenSource();
        var cancellationToken = cancellationTokenSource.Token;

        IPEndPoint ie = new IPEndPoint(IPAddress.Any, Port);

        using (Socket serverSocket = new Socket(AddressFamily.InterNetwork,
                                                SocketType.Stream,
                                                ProtocolType.Tcp))
        {
            serverSocket.Bind(ie);
            serverSocket.Listen(10);

            Console.WriteLine($"Servidor asíncrono iniciado");
            Console.WriteLine($"Escuchando en {ie.Address}:{Port}");
            Console.WriteLine("Esperando conexiones...");
        }
    }
}
```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

// Bucle con AcceptAsync en combinación
// con token de cancelación en vez de booleana simple.
try
{
    // Esta booleana se modifica con la función Cancel()
    // de cancellationTokenSource
    while (!cancellationToken.IsCancellationRequested)
    {

        // AcceptAsync: libera el hilo para ser reutilizado.
        // Otras tareas también pueden ejecutarse en paralelo.
        Socket clientSocket =
            await serverSocket.AcceptAsync(cancellationToken);

        // Usamos Task.Run en lugar de Thread
        // No usamos await, así la tarea se ejecuta en
        // segundo plano sin esperar a que acabe.
        // Se le pasa el cancellation Token para hacer también que finalice.
        _ = Task.Run(
            () => ClientDispatcherAsync(clientSocket, cancellationToken),
            cancellationToken
        );
    }
}
catch (OperationCanceledException)
{
    // Salta excepción si se cancela el token (no es un error)
    Console.WriteLine("Servidor detenido");
}

}

// Método asíncrono para manejar cliente
private async Task ClientDispatcherAsync(Socket sClient, CancellationToken cancellationToken)
{
    // Using asegura que el socket se cierra incluso si hay excepciones
    using (sClient)
    {

        IPEndPoint ieClient = (IPEndPoint)sClient.RemoteEndPoint;
        Console.WriteLine($"🌀 Cliente conectado: {ieClient.Address}:{ieClient.Port}");

        Encoding codificacion = Console.OutputEncoding;

        using (NetworkStream ns = new NetworkStream(sClient))
        using (StreamReader sr = new StreamReader(ns, codificacion))
        using (StreamWriter sw = new StreamWriter(ns, codificacion))
        {
            sw.AutoFlush = true;

            string welcome = "Welcome to The Echo-Logic, Odd, Desiderable, " +
                "Incredible, and Javaless Echo Async Server (T.E.L.O.D.I.J.E Async
Server)";

            // WriteLineAsync en lugar de WriteLine
            await sw.WriteLineAsync(welcome);

            string? message;

```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```

try
{
    // Bucle con cancelltationtoken nuevamente
    while (!cancellation_token.IsCancellationRequested)
    {
        // ReadLineAsync en vez de ReadLine
        // Permite cancelación mediante Token (Solo válido desde .Net6)
        message = await sr.ReadLineAsync().WaitAsync(cancellation_token);

        if (message == null)
        {
            // Cliente cerró la conexión normalmente
            Console.WriteLine($"Cliente {ieClient.Address} desconectado");
            break;
        }

        Console.WriteLine($"[{ieClient.Address}] dice: {message}");

        // Respuesta asíncrona
        await sw.WriteLineAsync($"Eco: {message}");
    }
} catch (OperationCanceledException)
{
    // la tarea de cliente al no tener await, es necesario controlar la excepción
    dentro.
    // El control externo es válido solo con await.
    Console.WriteLine($"Cliente {ieClient.Address} desconectado (servidor
detenido)");
} catch (IOException ex)
{
    // Error de conexión (cliente desconectado abruptamente)
    Console.WriteLine($"Cliente {ieClient.Address} desconectado abruptamente:
{ex.Message}");
}
}

// Método para detener el servidor de forma asíncrona
public async Task StopAsync()
{
    Console.WriteLine("Deteniendo servidor asíncrono...");

    // Cancelamos todas las operaciones pendientes
    cancellation_tokenSource?.Cancel();

    // Opcional: Esperar a que las tareas terminen (graceful shutdown)
    Console.WriteLine("Esperando que las conexiones terminen...");
    await Task.Delay(2000);

    Console.WriteLine("Servidor detenido completamente");
}

```

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios y procesos				CURSO: 2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:	
	UNIDAD COMPETENCIA					

```
// Método auxiliar para debugging
public int GetActiveThreadsCount()
{
    // Nota: En producción usaríamos una colección controlada de tareas
    return Process.GetCurrentProcess().Threads.Count;
}

// Programa principal adaptado
// Es importante indicar que al ser Task Main, espera a que acaben todas las tareas
// Recuerda que en un Main normal, las Task se tratan como Background y no esperaría,
// finalizando la aplicación.
internal class ProgramAsync
{
    static async Task Main(string[] args)
    {
        Console.WriteLine("=== Servidor Echo Asíncrono ===\n");

        var server = new EchoServerAsync();

        // Iniciamos el servidor de forma asíncrona
        var serverTask = Task.Run(() => server.InitServerAsync());

        // Espera para que no haya conflicto en mensajería
        await Task.Delay(500);

        // Recepción de comandos
        Console.WriteLine("Comandos disponibles:");
        Console.WriteLine("    status: Ver estado del servidor");
        Console.WriteLine("    stop: Detener servidor");

        bool running = true;
        while (running)
        {
            Console.Write("> ");
            string? command = Console.ReadLine();

            switch (command?.ToLower())
            {
                case "status":
                    Console.WriteLine($"Servidor en puerto {server.Port}");
                    Console.WriteLine($"Hilos en curso: {server.GetActiveThreadsCount()}");
                    Console.WriteLine("Estado: Ejecutándose");
                    break;

                case "stop":
                    await server.StopAsync();
                    running = false;
                    break;

                default:
                    Console.WriteLine("Comando no reconocido");
                    break;
            }
        }

        // Esperar a que el servidor termine completamente
        await serverTask;
        Console.WriteLine("\nPrograma terminado");
    }
}
```

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
UNIDAD COMPETENCIA							

En general hoy en día el uso más habitual para servidores en red que puedan atender a muchos clientes y sean escalables deben hacerse con Task. Se podría usar **Thread** en los siguientes casos:

- Aplicaciones simples con pocos clientes (< 50) sin necesidad de escalar. Un Thread ocupa alrededor de 1MB y un Task 1KB.
- Necesitas control total sobre prioridad/estado. En las Task no puedes gestionar la prioridad.
- Operaciones CPU-intensivas largas (procesamiento pesado).
- Aprendiendo conceptos básicos de concurrencia
- Versiones antiguas de .NET. Task solo hay desde la 4.5.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	UNIDAD COMPETENCIA							

Fuentes

C# Network Programming by Richard Blum. ISBN:0782141765. Sybex © 2003 (647 pages). Este libro a pesar de ser antiguo realmente viene muy completo y con muchos ejemplos del trabajo en red teniendo en cuenta que este no ha cambiado demasiado. Gran parte de ejemplos de este documento están basados en este libro.

TCP/IP Sockets in C#: Practical Guide for Programmers. David Makofske, Michael J. Donahoo, and Kenneth L. Calvert. ©2004 by Elsevier Inc. (Morgan Kauffman)

Visual C# 2010 Recipes: A Problem-Solution Approach © 2010 by Allen Jones and Adam Freeman. Apress.

Network Programming in .NET: With C# and Visual Basic .NET – 2022 Autor: Fiach Reid. 2ª edición (actualizada para .NET 6/7). Packt Publishing. ISBN: 978-1803233667

MS learn: Networking Programming

<https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/overview>

Wikipedia: <http://es.wikipedia.org> y <http://en.wikipedia.org>

StackOverflow: <https://stackoverflow.com>